僕たちのエラーレスポンスは間違っている

RFC 9457で脱・オレオレJSON

西 悠太　/ 株式会社ダイニー

自己紹介

西 悠太 (Nishi Yuta)

21歳(重要、important)

V8 Contributor

Platform Engineer at Dinii Inc.

TSKaigi Staff

TypeScriptが好きです
V8にパッチを送るのも好きです
あと野球も好きです

@riya-amemiya



エラーレスポンスの形
どうやって決めていますか？

オレオレJSON

エンドポイント	ステータス	ボディ
GET /users/42	404	`{ "error": "User not found" }`
POST /orders	400	`{ "message": "Out of stock", "code": 1001 }`
POST /signup	200	`{ "success": false, "errors": [{ "field": "email", "msg": "Invalid" }] }`

- APIが独自に決めたエラーレスポンスの形

クライアントのエラー処理

```
const res = await fetch(url, init)
const body = await res.json()

if (!res.ok || body.success === false) {
  const message =
    body.error ??
    body.message ??
    body.errors?.[0]?.msg ??
    "Something went wrong"
  showToast(message)
}
```

- クライアントはエンドポイントごとの形を全部知っていないといけない
- 形が1つ増えるたびに分岐も増える
- `1001` が何を表すかはドキュメントを読むまでわからない

ドキュメントの形骸化と実装の属人化

- エラーの形をエンドポイントごとにドキュメントへ書く
- 実装を変えてもドキュメントは古いまま
- エラーの形は、エンドポイントを書いた人が毎回決める

エラーの形に
標準はないの？

RFC 9457

Problem Details for HTTP APIs

2016年3月
RFC 7807

→

2023年7月
RFC 9457

- エラーの詳細をプログラムが読める形でレスポンスに載せる
- APIごとにエラーの形を定義しなくて済むようにする
- JSONのメディアタイプは `application/problem+json`

レスポンスの例

```
HTTP/1.1 403 Forbidden
Content-Type: application/problem+json
Content-Language: en

{
  "type": "https://example.com/probs/out-of-credit",
  "title": "You do not have enough credit.",
  "detail": "Your current balance is 30, but that costs 50.",
  "instance": "/account/12345/msgs/abc",
  "balance": 30,
  "accounts": ["/account/12345", "/account/67890"]
}
```

- RFC 9457にある、残高不足で購入できなかったときの例

ステータスコードとボディ

書く場所	読む側	伝えること
`HTTP/1.1 403 Forbidden`	プロキシ、キャッシュ HTTPライブラリ	拒否されたこと
ボディ	このAPIのクライアント	拒否された理由と残高

- 問題の種類はURIで識別する
- ボディに書くのはAPIの利用者に向けた詳細

それぞれのプロパティは
何を表している？

標準プロパティ

問題の種類ごとに決まる	`type`	問題の種類を識別するURI
	`title`	問題の種類の短い要約
発生ごとに変わる	`detail`	この発生についての説明
	`instance`	この発生を識別するURI
HTTPと同じ値	`status`	HTTPのステータスコード

type

- httpsのURIなら、開くとその問題のドキュメントが見られる
- tag URIのような開けないURIも使える
- あとから開けるURIに変えると、typeが変わって別の種類になる
- typeを省略すると `about:blank` になる

about:blank

```
{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404
}
```

- ステータスコード以上の意味がない問題を表す
- titleはステータスコードの文言と同じにする

titleとdetail

title

```
"You do not have enough credit."
```

種類ごとに同じ

detail

```
"Your current balance is 30, but that costs 50."
```

発生ごとに変わる

- titleを変えてよいのはAccept-Languageに合わせた翻訳だけ
- detailにはクライアントが問題を直すための説明を書く

instance

```
"instance": "/account/12345/msgs/abc"
```

- 問い合わせを受けたとき、どの発生の話かを特定できる
- 開けるURIなら、この発生の詳細を取得できる
- 開けないURIなら、サーバーだけが意味を知るIDとして使う

status

```
HTTP/1.1 403 Forbidden
Content-Type: application/problem+json

{
  "type": "https://example.com/probs/out-of-credit",
  "status": 403
}
```

- 実際のステータスコードと必ず同じ値にする
- 中継やキャッシュでステータスコードが変わっても元の値がわかる
- ボディだけを保存しても元の値がわかる

残高や入金先のような情報は
どこに入れる？

拡張メンバー

```
{
  "type": "https://example.com/probs/out-of-credit",
  "title": "You do not have enough credit.",
  "detail": "Your current balance is 30, but that costs 50.",
  "instance": "/account/12345/msgs/abc",
  "balance": 30,
  "accounts": ["/account/12345", "/account/67890"]
}
```

- 標準プロパティのほかに、問題の種類ごとに足すプロパティ
- `balance` は残高、`accounts` は入金できる口座へのリンク
- クライアントはdetailを解析せずに値を受け取れる

安全な拡張



- クライアントは知らない拡張メンバーを必ず無視する
- 名前は英字で始め、英数字と `_` で3文字以上にする

複数のエラー

リクエスト

```
{
  "age": 42.3,
  "profile": {
    "color": "yellow"
  }
}
```

レスポンス 422

```
{
  "type": "https://example.net/validation-error",
  "title": "Your request is not valid.",
  "errors": [
    {
      "detail": "must be a positive integer",
      "pointer": "#/age"
    },
    {
      "detail": "must be 'green', 'red' or 'blue'",
      "pointer": "#/profile/color"
    }
  ]
}
```

- 同じ種類の問題は `errors` にまとめる
- `pointer` はリクエストの中の場所をJSON Pointerで指す
- 種類が違う問題が重なったら、一番急ぎか重要な1つだけ返す

新しい問題の種類を作るとき

- PUTへの403のように、ステータスコードで足りるなら作らない
- IANAのレジストリに合う種類があれば、それを使う
- type URI、title、ステータスコードを必ずドキュメントに書く
- type URIの先に、解決方法を説明するHTMLを置く

フロントエンドとバックエンドの境界でどう運用する？

バックエンドの実装

```
type ProblemDetails = {
  type?: string
  title?: string
  status?: number
  detail?: string
  instance?: string
  [extension: string]: unknown
}

const problem = (body: ProblemDetails & { status: number }) =>
  new Response(JSON.stringify(body), {
    status: body.status,
    headers: { "Content-Type": "application/problem+json" },
  })
```

- ステータスコードと `status` を同じ値から作る

バックエンドで守ること

- スタックトレースのような実装の内部情報を返さない
- typeとinstanceは絶対URIで書く
- 既存のAPIはAcceptヘッダーを見て新旧の形を返し分ける

フロントエンドで守ること

- Content-Typeが `application/problem+json` か確かめてから読む
- typeで分岐し、titleやdetailの文字列では分岐しない
- 知らないtypeと `about:blank` はステータスコードで扱う
- typeのURIを自動で開きに行かない
- 入力エラーは `errors` の `pointer` でフォームの項目に対応づける

フロントエンドの実装

```
const res = await fetch("/purchase", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    Accept: "application/json, application/problem+json",
  },
  body: JSON.stringify({ item: 123456, quantity: 2 }),
})

if (res.headers.get("Content-Type")?.startsWith("application/problem+json")) {
  const problem: ProblemDetails = await res.json()
  switch (problem.type ?? "about:blank") {
    case "https://example.com/probs/out-of-credit":
      showTopUp(problem)
      break
    default:
      showErrorFor(res.status)
  }
}
```

次に作るエラーレスポンスは
application/problem+jsonで返そう

参考資料

- RFC 9457: Problem Details for HTTP APIs
- RFC 7807: Problem Details for HTTP APIs
- IANA: HTTP Problem Types
- RFC 9110: HTTP Semantics
- RFC 6901: JavaScript Object Notation (JSON) Pointer

ご清聴ありがとうございました